

使用偏好媒體查詢，建構可配合使用者調整的介面

來源：<https://codelabs.developers.google.com/codelabs/user-adaptive-interfaces?hl=zh-tw>

步驟數：7

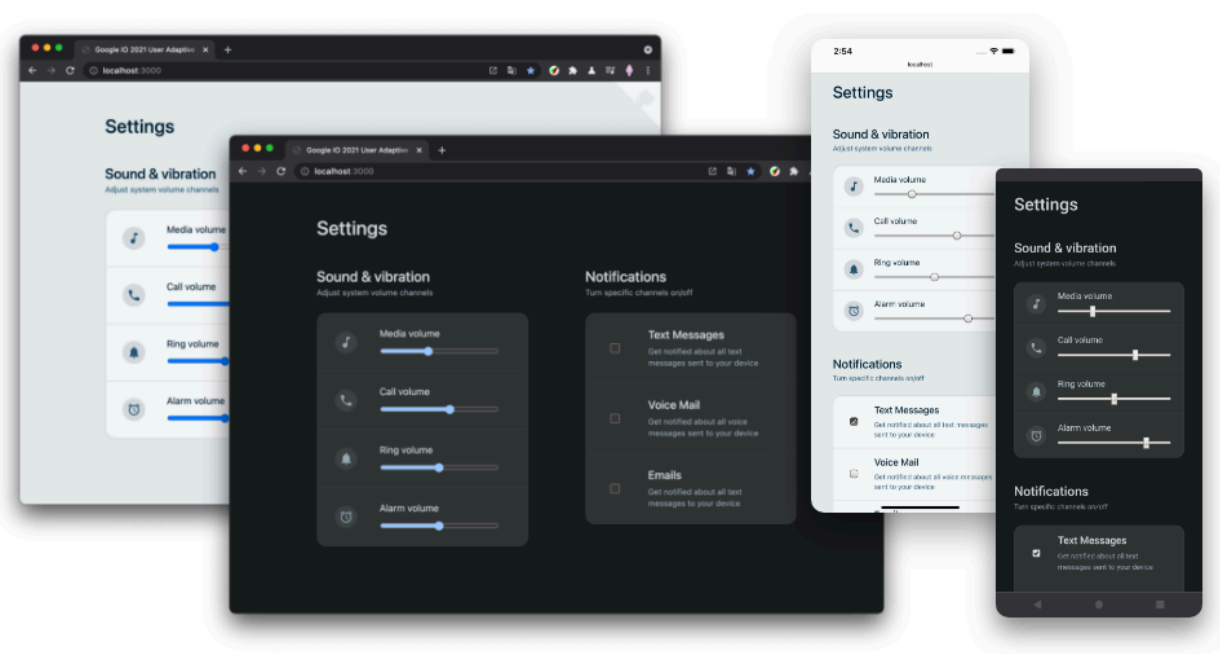
瞭解如何建立自我調適網頁表單，讓每位使用者都會收到可反映特定時間偏好設定的專屬介面。

目錄

- 1. [1. 事前準備](#)
- 2. [2. 做好準備](#)
- 3. [3. 自動調整互動](#)
- 4. [4. 自動調整式版面配置](#)
- 5. [5. 自動調整顏色](#)
- 6. [6. 自動調整動畫](#)
- 7. [7. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 事前準備



如今，使用者會在裝置上指出許多[偏好設定](#)。他們希望作業系統和應用程式的外觀和風格與自己的裝置一致。使用者適應型介面會運用這些偏好設定提升使用者體驗，讓使用者感覺更[貼近個人需求](#)。如果做法正確，使用者可能永遠不會知道使用者體驗正在調整或已調整。

使用者偏好設定

裝置硬體是偏好設定，作業系統是選擇，應用程式和作業系統顏色是偏好設定，應用程式和作業系統文件語言也是偏好設定。使用者偏好設定的數量似乎只會增加。網頁無法存取所有內容，[這是基於正當理由](#)。

以下列舉幾個 CSS 可使用的使用者偏好設定：

- [裝置可視區域](#)大小
- [裝置螢幕方向](#) (直向 | 橫向)
- 字型大小
- [線上 / 離線](#)
- [網路品質](#)
- [色彩配置](#) (淺色 | 深色)
- [介面動畫](#) (開啟 | 減少)
- [輸入品質](#) (滑鼠 | 觸控 | 觸控筆)
- [文件方向和書寫模式](#) (從左到右、從上到下等)
- [顯示模式](#) (全螢幕 | 獨立 | 最小化 UI | 瀏覽器)

以下列舉幾個[即將在 CSS 中推出](#)的使用者偏好設定：

- [減少數據用量](#) / 精簡模式
- [色域](#)
- [對比度](#) (降低 | 提高 | 減少)
- [顏色](#) (強制將使用者的顏色套用至網頁)
- [環境音](#) (開啟 | 減少)

媒體查詢

CSS 和網路可透過媒體查詢 (一種宣告式條件，如果該條件為 true，則包含一組樣式) 進行調整和回應。最常見的情況是裝置的可視區域大小條件：如果大小小於 800 像素，則適用於這種情況的樣式會更好。

使用者適應性

如果介面無法適應，使用者造訪時不會有任何變化，也就是說，所有使用者都會獲得相同的體驗，且無法調整。使用者適應型介面可能會為五位不同的使用者提供五種不同的外觀和樣式。功能相同，但使用者可調整使用者介面，因此介面美觀度較佳，也更易於使用。

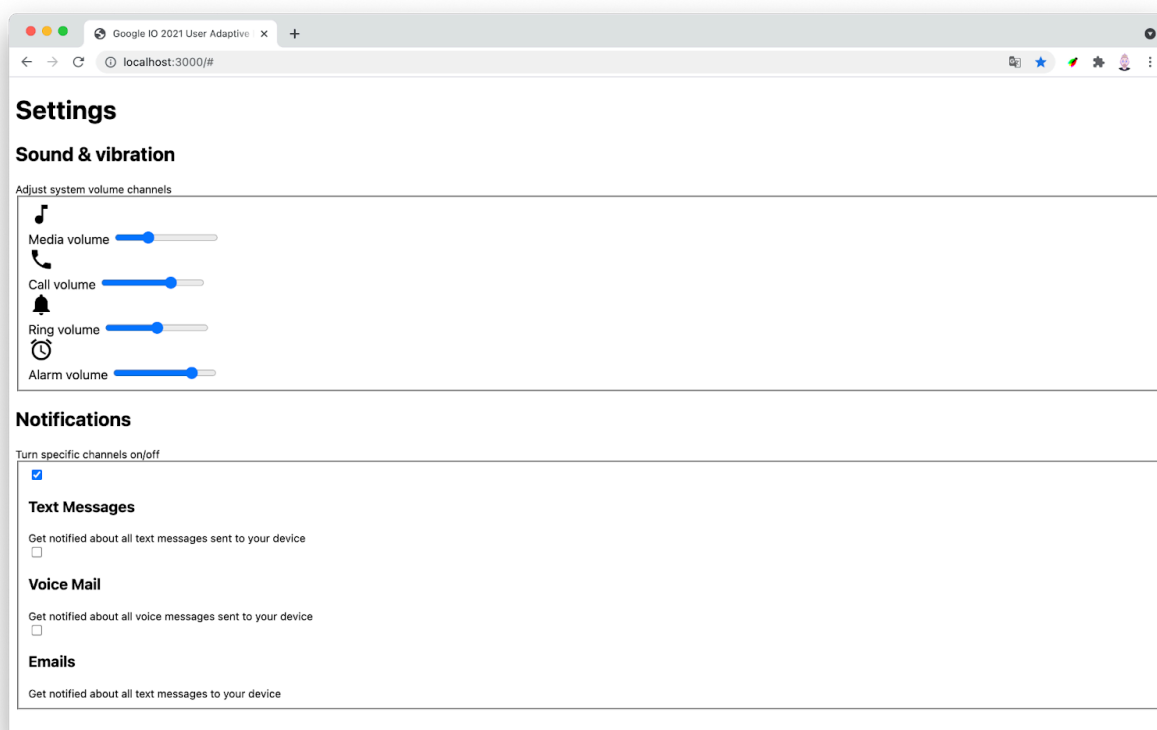
必要條件

- 瞭解 HTML 和 CSS
- 熟悉開發人員工具，例如 [Google Chrome 開發人員工具](#)

建構項目

在本程式碼研究室中，您將建構可適應下列情況的使用者適應性表單：

- 系統色彩配置偏好設定，可為表單控制項和周圍的 UI 元素提供淺色和深色色彩配置
- 系統動作偏好設定，提供多種動畫類型
- 大小裝置的檢視區塊，提供行動裝置和電腦體驗
- 多種輸入類型，例如鍵盤、螢幕閱讀器、觸控和滑鼠
- 任何語言和讀取/寫入模式

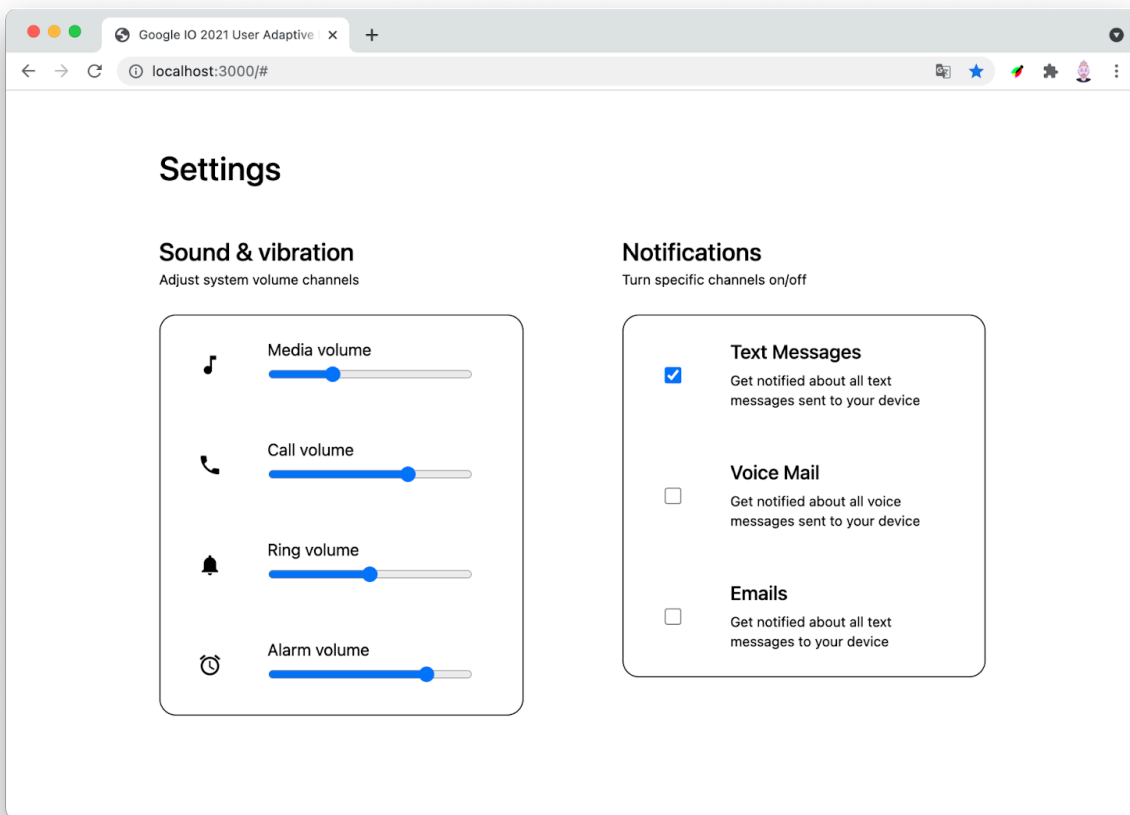


課程內容

在本程式碼研究室中，您將瞭解現代網頁功能，協助您建構可自動調整版面配置的表單。您會瞭解如何執行下列作業：

- 製作淺色和深色主題

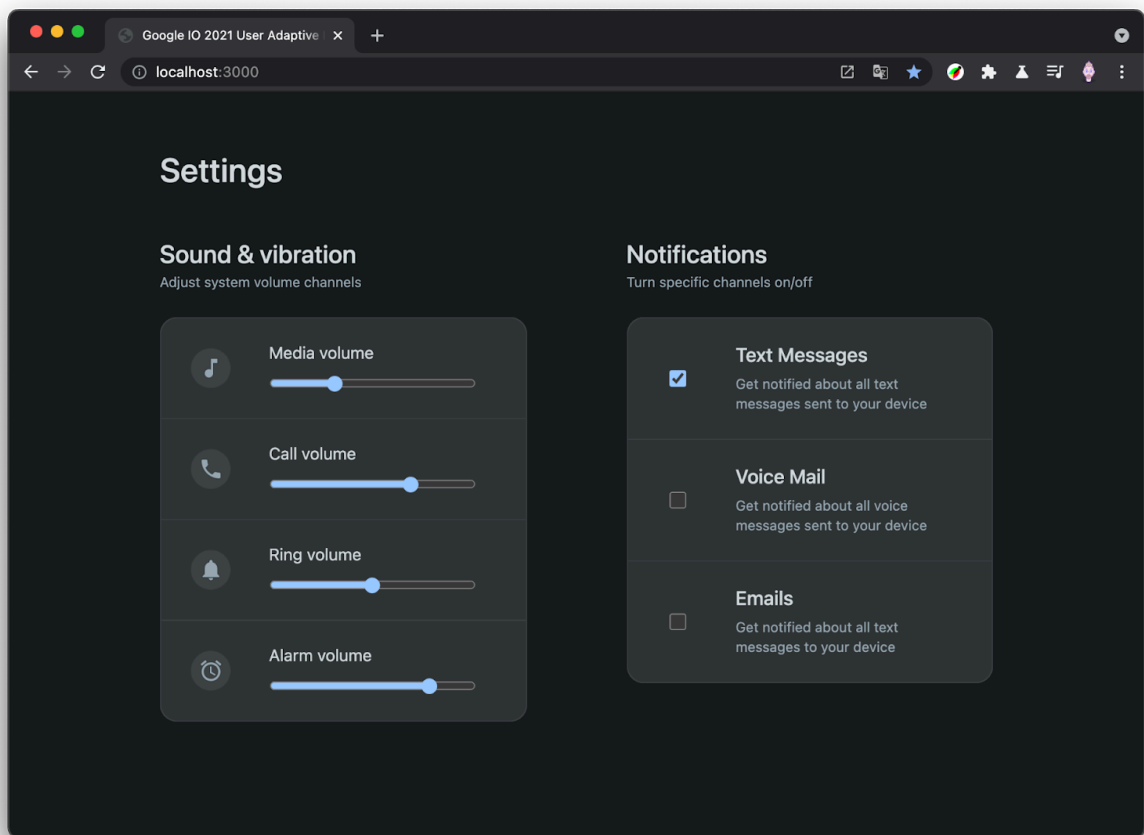
- 建立動畫和無障礙表單
- 版面配置回應式表單
- 使用相對單位和邏輯屬性



本程式碼研究室著重於自動調整式使用者介面。我們不會對與主題無關的概念和程式碼多做介紹，但會事先準備好這些程式碼區塊，屆時您只要複製及貼上即可。

軟硬體需求

- Google Chrome 89 以上版本，或偏好的瀏覽器



步驟 2 · 約 2 分鐘

2. 做好準備

取得程式碼

本專案所需的一切資料都放在 [GitHub 存放區](#) 中。如要開始進行本專案，請先取得程式碼並在慣用的開發人員環境中開啟。或者，您也可以分叉 [這個 Codepen](#)，然後從該處開始作業。

建議：使用 Codepen

1. 開啟新的瀏覽器分頁。
2. 前往 <https://codepen.io/argyleink/pen/abBMeeq>。
3. 如果沒有帳戶，請建立帳戶來儲存工作。
4. 按一下「Fork」(分叉)。

替代方案：在本機工作

如要下載程式碼並在本機作業，您需要 [Node.js](#) 12 以上版本，以及設定完成且可供使用的程式碼編輯器。

使用 Git

1. 前往 https://github.com/argyleink/Google-IO-2021-Workshop_User-Adaptive-Interfaces
2. 將存放區複製到資料夾。
3. 請注意，預設分支版本為 `beginning`。

使用檔案

1. 將下載的 ZIP 檔案解壓縮至資料夾。

執行專案

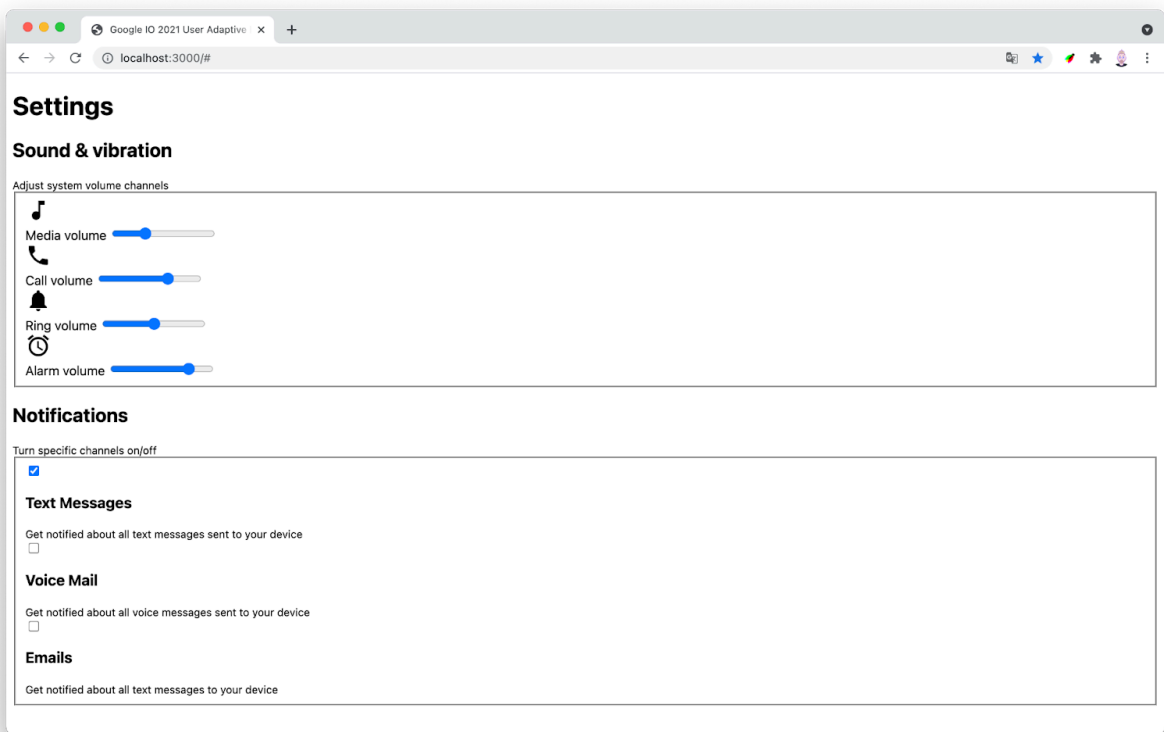
使用上述任一步驟中建立的專案目錄，然後：

1. 執行 `npm install`，安裝執行伺服器所需的依附元件。
2. 執行 `npm start`，在通訊埠 3000 啟動伺服器。
3. 開啟新的瀏覽器分頁。
4. 前往 <http://localhost:3000>。

關於 HTML

本課程將介紹用於支援使用者適應性互動的 HTML 相關層面。本研討會特別著重於 CSS。如果您是表單或網站建構新手，建議查看提供的 HTML。就無障礙和版面配置而言，HTML 元素的選擇至關重要。

準備好開始後，您會將這個架構轉換為動態自動調整式使用者體驗。



3. 自動調整互動

Git 分支：`beginning`

本節結束時，您的設定表單將調整為：

- 遊戲手把 + 鍵盤
- 滑鼠 + 觸控
- 螢幕閱讀器或類似的輔助技術

HTML 的屬性

來源程式碼中提供的 HTML 是絕佳的起點，因為系統已選擇語意元素，協助您將表單輸入內容分組、排序及加上標籤。

表單通常是商家與使用者互動的關鍵點，因此表單必須能適應網路可提供的多種輸入類型。舉例來說，您可能需要提供可透過觸控操作的行動裝置專用表單。在本節中，您會在版面配置和樣式之前，確保自適應輸入的可用性。

分組輸入內容

HTML 中的 `<fieldset>` 元素可將類似的輸入內容和控制項分組。表單中有兩個群組，分別是音量和通知。這對使用者體驗很重要，因為使用者可以略過整個區段。

排序元素

元素順序會依邏輯順序提供。這對使用者體驗非常重要，因為遊戲手把、鍵盤或螢幕閱讀器技術的視覺體驗順序相同或相似。

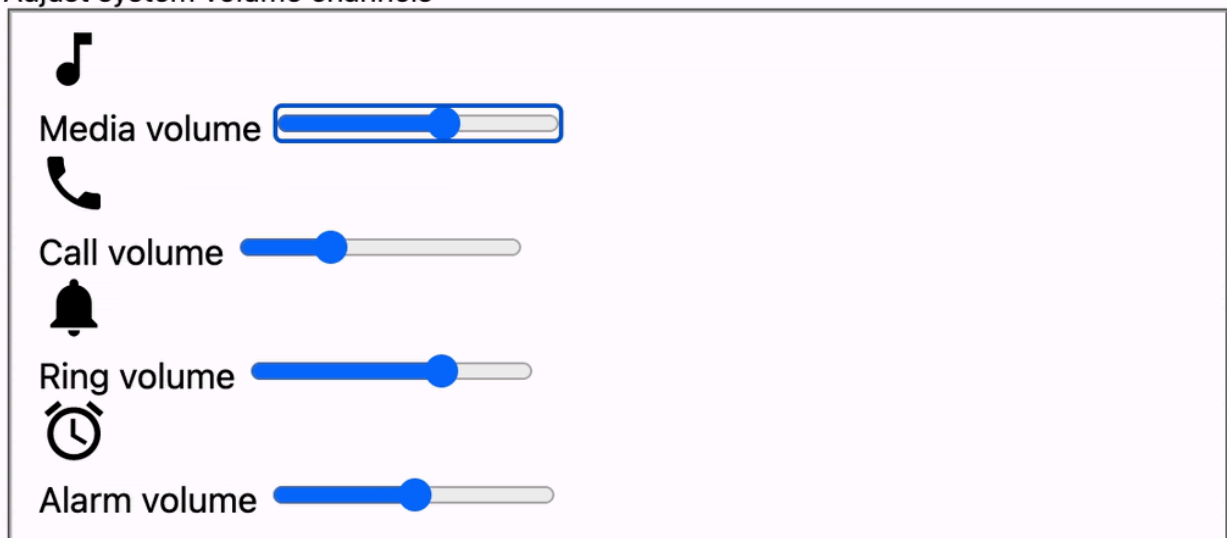
鍵盤互動

網路使用者已習慣使用 Tab 鍵在表單中移動，幸好只要提供預期的 HTML 元素，瀏覽器就會處理這項操作。使用

`<button>`、`<input>`、`<h2>` 和 `<label>` 等元素時，這些元素會自動成為鍵盤或螢幕閱讀器目的地。

Sound & vibration

Adjust system volume channels



上方影片示範如何使用 Tab 鍵和箭頭鍵在介面中移動，並進行變更。不過，藍色外框緊緊圍繞輸入元素，請新增下列樣式，讓這個互動留一點空間。

style.css

```
input {  
  outline-offset: 5px;  
}
```

建議做法

1. 查看 `index.html` 中使用的 HTML 元素。
2. 在瀏覽器中點選示範網頁。
3. 按下 `tab` 鍵和 `shift+tab` 鍵，即可在表單中移動元素焦點。
4. 使用鍵盤變更滑桿和核取方塊的值。
5. 連線至藍牙遊戲手把控制器，然後透過表單移動元素焦點。

滑鼠互動

網路使用者已習慣使用滑鼠與表單互動。請嘗試在表單上使用滑鼠。滑桿和核取方塊可以運作，但仍有改善空間。滑鼠點選這些核取方塊時，可能會覺得太小。

Turn specific channels on/off

☒

Text Messages

Get notified about all text messages sent to your device

☐

Voice Mail

Get notified about all voice messages sent to your device

☐

Emails

Get notified about all text messages to your device

瞭解如何取得兩項使用者體驗功能，連結標籤及其輸入內容？

第一個功能是提供互動選項，且標籤比小方塊更容易以滑鼠點選。

第二項功能是確切知道標籤適用於哪個輸入內容。如果沒有 CSS，您很難判斷哪個標籤對應哪個核取方塊，除非您提供一些屬性。

這種明確的連結也能提升螢幕閱讀器的使用體驗，下一節將介紹這項工具。

未建立關聯：沒有任何屬性可連結元素

```
<input type="checkbox">  
<label>...</label>
```

關聯：連結元素的屬性

```
<input type="checkbox" id="voice-notifications" name="voice-notifications">
<label for="voice-notifications">...</label>
```

提供的 HTML 已歸因所有輸入內容和標籤。如果您不熟悉這個概念，建議進一步瞭解。

建議做法

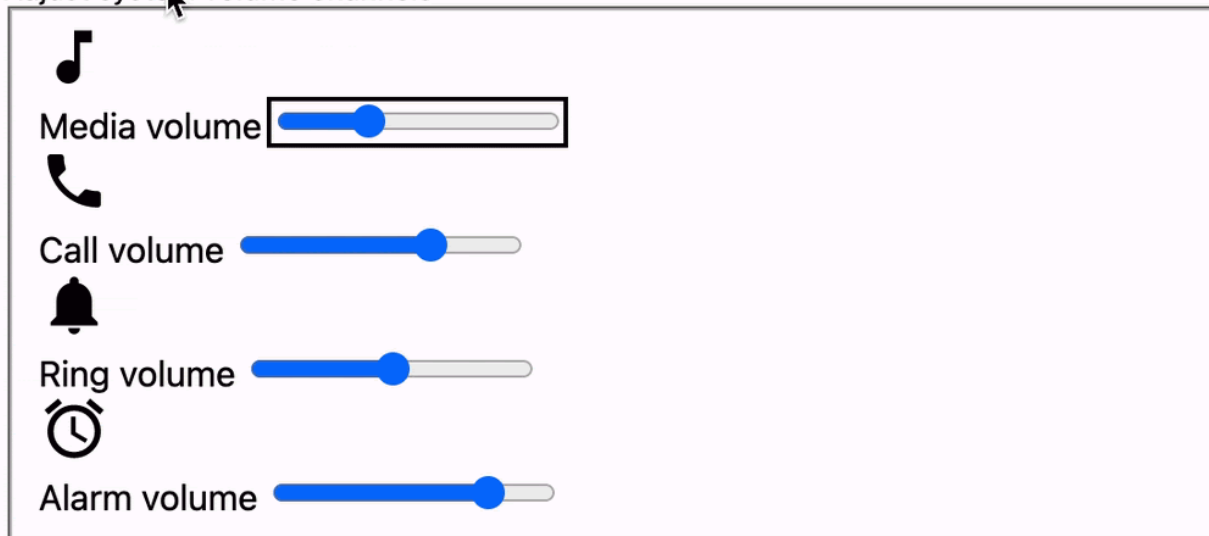
1. 將滑鼠懸停在標籤上，你會發現核取方塊會醒目顯示。
2. 使用 Chrome 開發人員工具調查標籤元素，以便顯示可選取核取方塊的可點選表面區域。

螢幕閱讀器互動

輔助技術可以與這個表單互動，只要使用幾個 HTML 屬性，就能讓使用者體驗更順暢。

Sound & vibration

Adjust system volume channels



如果使用者在 Chrome 中使用螢幕閱讀器瀏覽目前的表單，系統會在 `<picture>` 元素中不必要地停止 (這並非 Chrome 特有的問題)。使用螢幕閱讀器的使用者可能因為視力障礙而使用這項工具，因此停留在圖片上對他們沒有幫助。你可以使用屬性，向螢幕閱讀器隱藏元素。

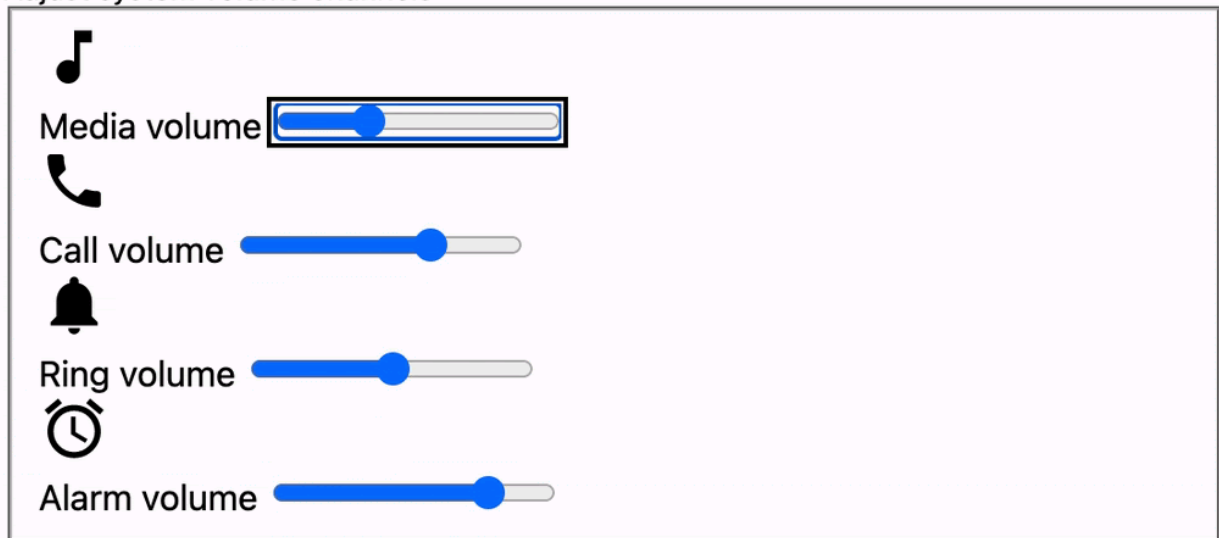
index.html

```
<picture aria-hidden="true">
```

現在螢幕閱讀器會略過純視覺元素。

Sound & vibration

Adjust system volume channels



滑桿元素 `input[type="range"]` 具有特殊的 [ARIA](#) 屬性：`aria-labelledby="media-volume"`。這項屬性會為螢幕閱讀器提供特殊指令，以提升使用者體驗。

建議做法

1. 使用作業系統的螢幕閱讀器技術，在表單中移動焦點。
2. 下載並試用表單上的螢幕閱讀器軟體。

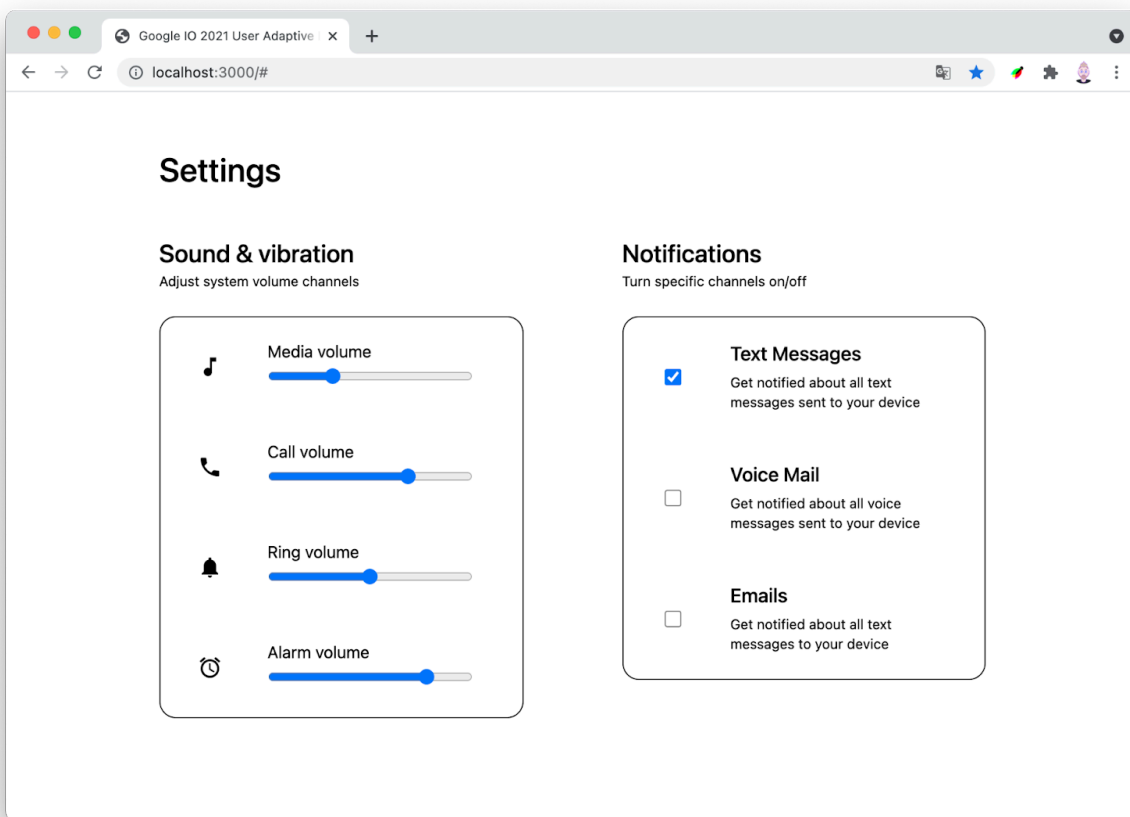
步驟 4 · 約 10 分鐘

4. 自動調整式版面配置

Git 分支版本：`layouts`

完成本節後，「設定」頁面將：

- 使用自訂屬性和使用者相對單位建立間距系統
- 撰寫 CSS 格線，實現彈性且具回應性的對齊和間距
- 使用邏輯屬性，打造可適應國際市場的版面配置
- 編寫媒體查詢，在精簡和寬敞的版面配置之間調整



間距

如要打造美觀版面配置，請善用有限的間距選項。這有助於內容自然對齊和諧音。

自訂屬性

本研討會以一組七種自訂屬性大小為基礎。

- 將這些內容放在 `style.css` 頂端：

style.css

```
:root {
  --space-xs: .25rem;
  --space-xs: .5rem;
  --space-sm: 1rem;
  --space-md: 1.5rem;
  --space-lg: 2rem;
  --space-xl: 3rem;
  --space-xxl: 6rem;
}
```

這些名稱與人們彼此用來描述空間的用語相近。您也可以專門使用 `rem` 單位，確保單位大小清晰易讀，並配合使用者偏好設定調整。

網頁樣式

接著，您需要設定一些文件樣式、移除元素邊界，並將字型設為美觀的無襯線字型。

- 在 `style.css` 中新增下列項目：

style.css

```
* {
  box-sizing: border-box;
  margin: 0;
}

html {
  block-size: 100%;
}

body {
  min-block-size: 100%;
  padding-block-start: var(--space-xs);
  padding-block-end: var(--space-xs);
}
```

這就是您第一次使用間距自訂屬性！這就是太空之旅的起點。

Typography

這個版面配置的字型是自適應字型。 `system-ui` 關鍵字會使用使用者作業系統決定的最佳介面字型。

```
body {
  font-family: system-ui, sans-serif;
}

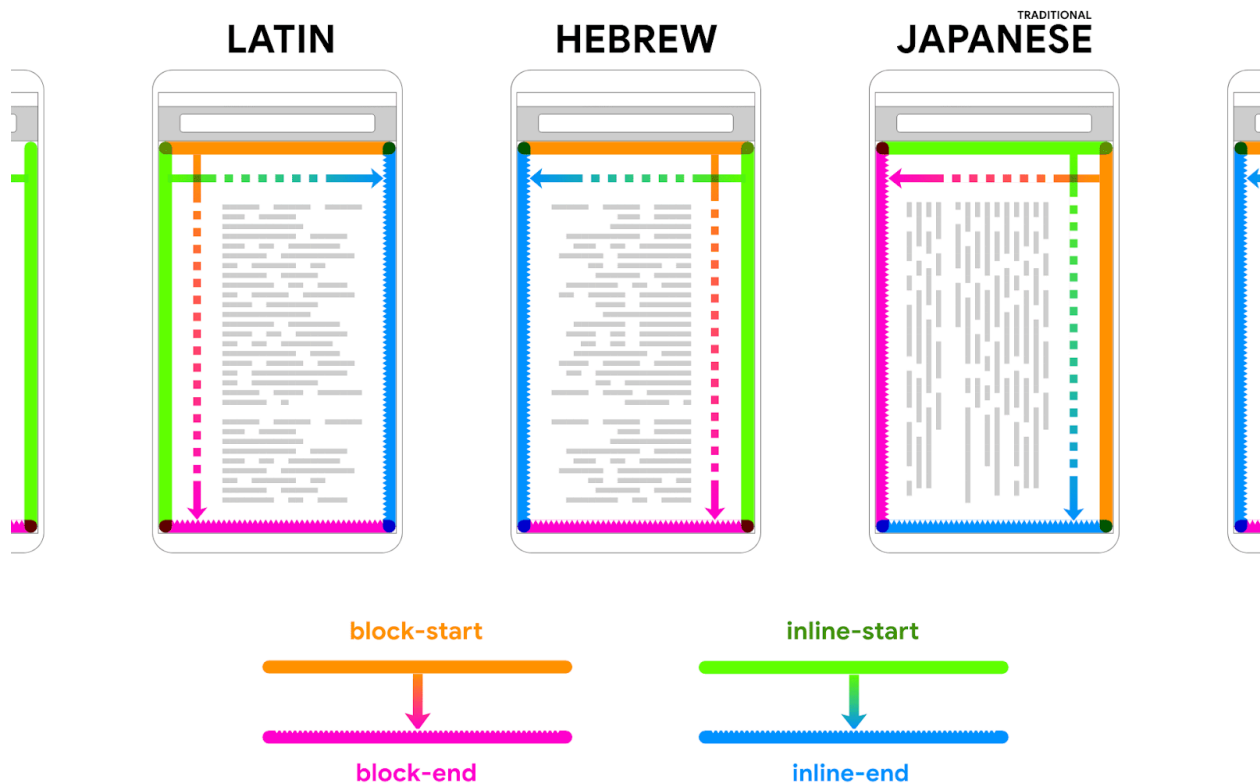
h1,h2,h3 {
  font-weight: 500;
}

small {
  line-height: 1.5;
}
```

h1、h2 和 h3 的樣式是次要的，且具有風格。不過，如果文字會換行，`small` 元素就需要額外的 `line-height`。否則會過於擁擠。

邏輯屬性

請注意，`padding` 使用**邏輯屬性** (`block-start`、`block-end`) 指定側邊。`body` 在程式碼研究室的其餘部分，我們將大量使用邏輯屬性。與 `rem` 單位一樣，這類廣告也會配合使用者調整。這個版面配置可以翻譯成其他語言，並根據使用者習慣的母語，設定自然書寫和文件方向。只要定義一次空間、方向或對齊方式，邏輯屬性就能支援這項功能。



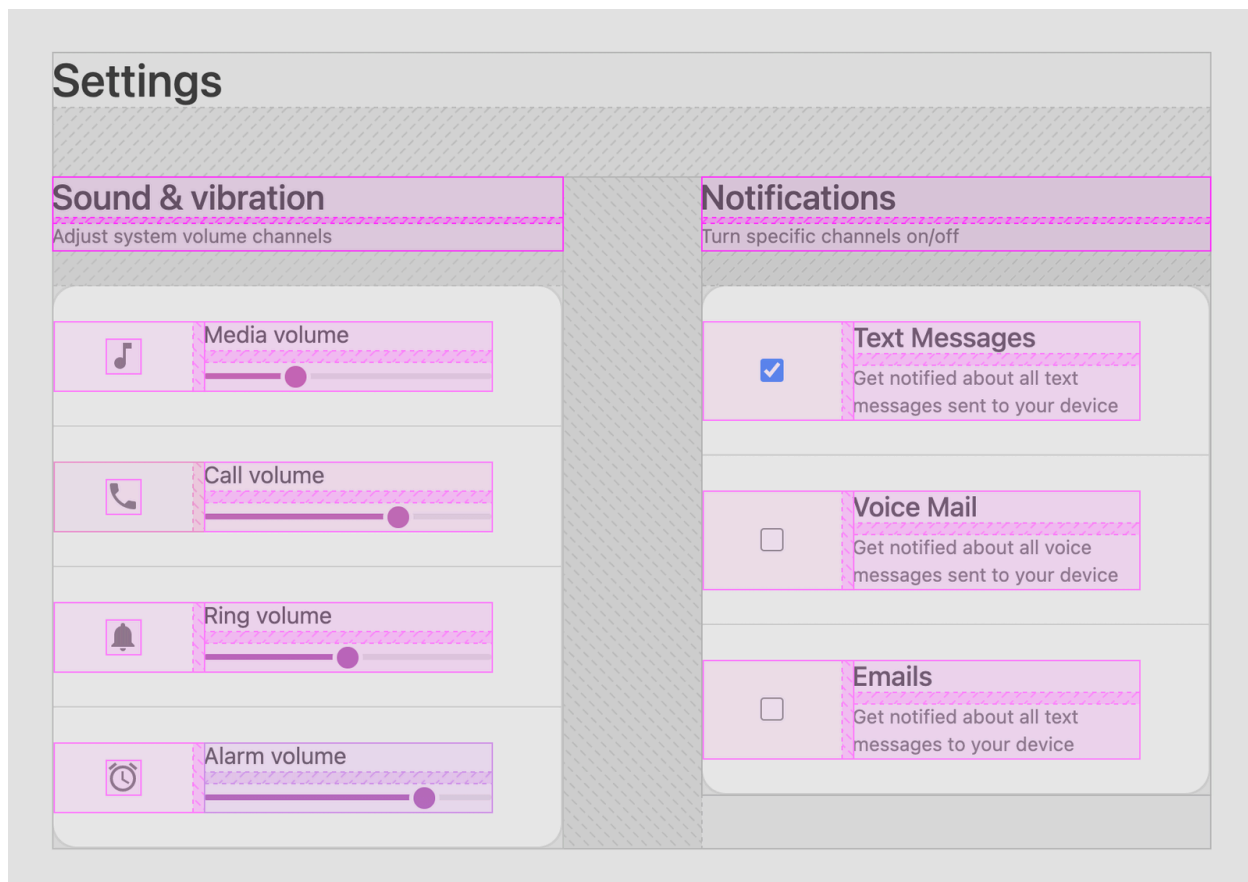
格線和彈性方塊已是[流程相關](#)，也就是說，為某種語言編寫的樣式會根據脈絡，適當套用至其他語言。自動調整方向性；內容會根據文件方向性流動。

使用邏輯屬性，您就能以較少的樣式觸及更多使用者。

CSS 格線版面配置

grid CSS 屬性是功能強大的版面配置工具，提供許多功能來處理複雜工作。您將建構幾個簡單的格狀版面配置和一個複雜的版面配置。您也會由外而內地工作，從巨集版面配置到微版面配置。間距自訂屬性將變得至關重要，因為它們不僅是內距或邊界值，也是欄大小、邊框半徑等。

以下是 Chrome 開發人員工具一次疊加顯示各個 CSS 格線版面的螢幕截圖：



1. 請將下列各項樣式新增至 `style.css`，以便跟著操作：

<main>

```
main {  
  display: grid;  
  gap: var(--space-xl);  
  place-content: center;  
  padding: var(--space-sm);  
}
```

根據預設，每個子項元素都會放入自己的列中，因此非常適合堆疊元素。此外，使用 `gap` 也有額外好處。先前，您已使用 `*` 選取器在所有元素上設定 `margin: 0`，這項設定現在很重要，因為您要使用 `gap` 設定間距。間隙不僅是管理容器中空間的單一位置，也是相對流程。

<form>


```
form {
  max-width: 89vw;
  display: grid;
  gap: var(--space-xl) var(--space-xxl);
  align-items: flex-start;
  grid-template-columns:
    repeat(auto-fit, minmax(min(10ch, 100%), 35ch));
}
```

這是設計中最棘手的格線版面配置，但考量了最令人興奮的回應式層面：

- `max-width` 提供版面配置演算法使用的值，以決定大小。
 - `gap` 使用自訂屬性，並從 `column-gap` 傳遞不同的 `row-gap`。
 - `align-items` 設為 `flex-start`，以免項目高度遭到延展。
 - `grid-template-columns` 的語法較為複雜，但目標是簡潔；請將欄寬設為 `35ch`，且不得小於 `10ch`，並在有空間時將項目放入欄中，否則請使用列。
2. 測試調整瀏覽器大小。請注意，表單會在小型可視區域中收合為列，但如果空間足夠，則會流入新的欄，調整大小，不需要使用媒體查詢。這種不含媒體查詢的自適應樣式策略，特別適合用於元件或以內容為中心的版面配置。

<section>

```
section {
  display: grid;
  gap: var(--space-md);
}
```

每個區段都應是列格線，子元素之間有中等間距。

```
header {
  display: grid;
  gap: var(--space-xs);
}
```

每個標題都應是列格線，子項元素之間有極小的額外空間。

<fieldset>

```
fieldset {
  padding: 0;
  display: grid;
  gap: 1px;
  border-radius: var(--space-sm);
  overflow: hidden;
}
```

這個版面配置負責建立類似資訊卡的樣式，並將輸入內容歸類在一起。在下一節中新增顏色後，`overflow: hidden` 和 `gap: 1px` 就會變得清楚。

.fieldset-item

```
.fieldset-item {
  display: grid;
  grid-template-columns: var(--space-lg) 1fr;
  gap: var(--space-md);
  padding: var(--space-sm) var(--space-md);
}
```

這個版面配置負責將圖示和核取方塊與相關聯的標籤和控制項置中對齊。格線範本的第一欄 `var(--space-lg)` 會建立比圖示寬的欄，因此子項元素可以在其中置中。

這個版面配置會顯示自訂屬性中已做出的設計決策數量。使用您已定義的值，在系統的協調性內調整邊框間距、間隙和資料欄的大小。

.fieldset-item <picture>

```
.fieldset-item > picture {
  block-size: var(--space-xl);
  inline-size: var(--space-xl);
  clip-path: circle(50%);
  display: inline-grid;
  place-content: center;
}
```

這個版面配置負責設定、圖示圓圈的大小、建立圓形形狀，以及將圖片置中於圓圈內。

<picture> 和 [checkbox] 對齊

```
.fieldset-item > :is(picture, input[type="checkbox"]) {
  place-self: center;
}
```

這個版面配置使用 `:is` [虛擬選取器](#)，將圖片和核取方塊元素置中。

3. 將選取器 `picture > svg` 替換為 `.fieldset-item svg`，如下所示：

.fieldset-item <svg>

```
.fieldset-item svg {
  block-size: var(--space-md);
}
```

這會將 SVG 圖示大小設為尺寸系統中的值。

.sm-stack

```
.sm-stack {
  display: grid;
  gap: var(--space-xs);
}
```

這個公用程式類別適用於核取方塊標籤元素，可為核取方塊的輔助文字設定間距。

input[type="checkbox"]

```
input[type="checkbox"] {
  inline-size: var(--space-sm);
  block-size: var(--space-sm);
}
```

這些樣式會使用間距集中的值，增加核取方塊的大小。

建議做法

1. 開啟 Chrome 開發人員工具，然後在「元素」面板的 HTML 中找出格線徽章。按一下即可開啟偵錯工具。
2. 開啟 Chrome 開發人員工具，然後將滑鼠游標懸停在「樣式」窗格中的「間隙」上。
3. 開啟 Chrome 開發人員工具，前往「樣式」窗格，然後從「樣式」切換至「版面配置」。切換設定並開啟版面配置，即可探索這個區域。

媒體查詢

下列 CSS 會根據可視區域大小和方向調整樣式，目的是根據可視區域環境調整間距或排列方式，達到最佳效果。

注意：各部分的 CSS 應在 `style.css` 中，新增至要修改的選取器後方。

<main>

```
@media (min-width: 540px) {
  main {
    padding: var(--space-lg);
  }
}

@media (min-width: 800px) {
  main {
    padding: var(--space-xl);
  }
}
```

這兩項媒體查詢會在有更多檢視區塊空間時，提供 `main` 更多邊框間距。也就是說，一開始會以少量的精簡邊框間距呈現，但隨著可用空間增加，邊框間距會越來越大。

<form>

```
form {
  --repeat: auto-fit;
  grid-template-columns:
    repeat(var(--repeat), minmax(min(10ch, 100%), 35ch));
}

@media (orientation: landscape) and (min-width: 640px) {
  form {
    --repeat: 2;
  }
}
```

表單已透過 `auto-fit` 回應可視區域大小，但在行動裝置上測試時，將裝置轉為橫向模式並不會讓兩個表單群組並排顯示。使用 `orientation` 媒體查詢和可視區域寬度檢查，調整為這個橫向環境。現在，如果裝置為橫向且寬度至少為 640 像素，請將 `--repeat` 自訂屬性切換為數字 (而非 `auto-fit` 關鍵字)，強制使用兩欄。

.fieldset-item

```
@media (min-width: 540px) {
  .fieldset-item {
    grid-template-columns: var(--space-xxl) 1fr;
    gap: var(--space-xs);
    padding: var(--space-md) var(--space-xl) var(--space-md) 0;
  }
}
```

如果可視區域空間充足，這項媒體查詢會進一步擴大間距。格線範本會使用範本中較大的自訂屬性 (`var(--space-xxl)`)，擴展第一欄。邊框間距也會擴大為較大的自訂屬性。

建議做法

1. 展開及收合瀏覽器，並觀察空間的調整情形。
 2. 在行動裝置上預覽
 3. 在行動裝置上以橫向模式預覽
-

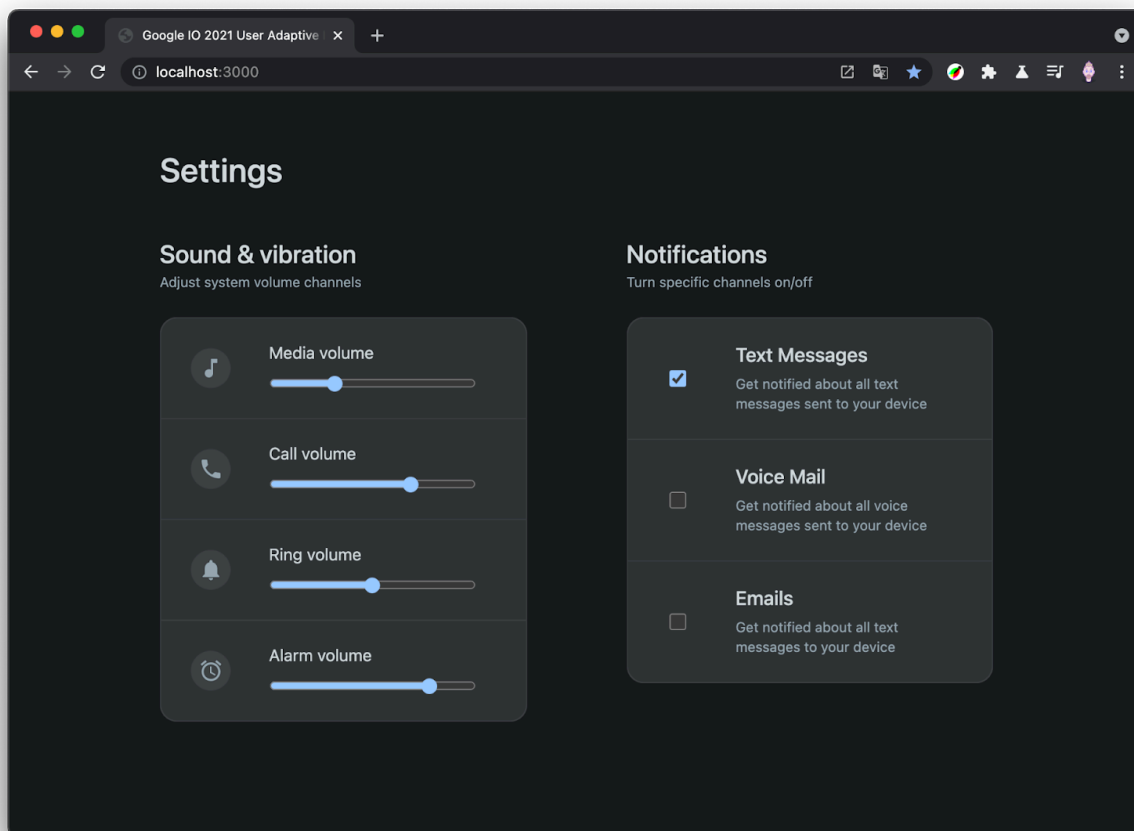
步驟 5 · 約 10 分鐘

5. 自動調整顏色

Git 分支版本：`colors`

本節結束時，您的設定表單將：

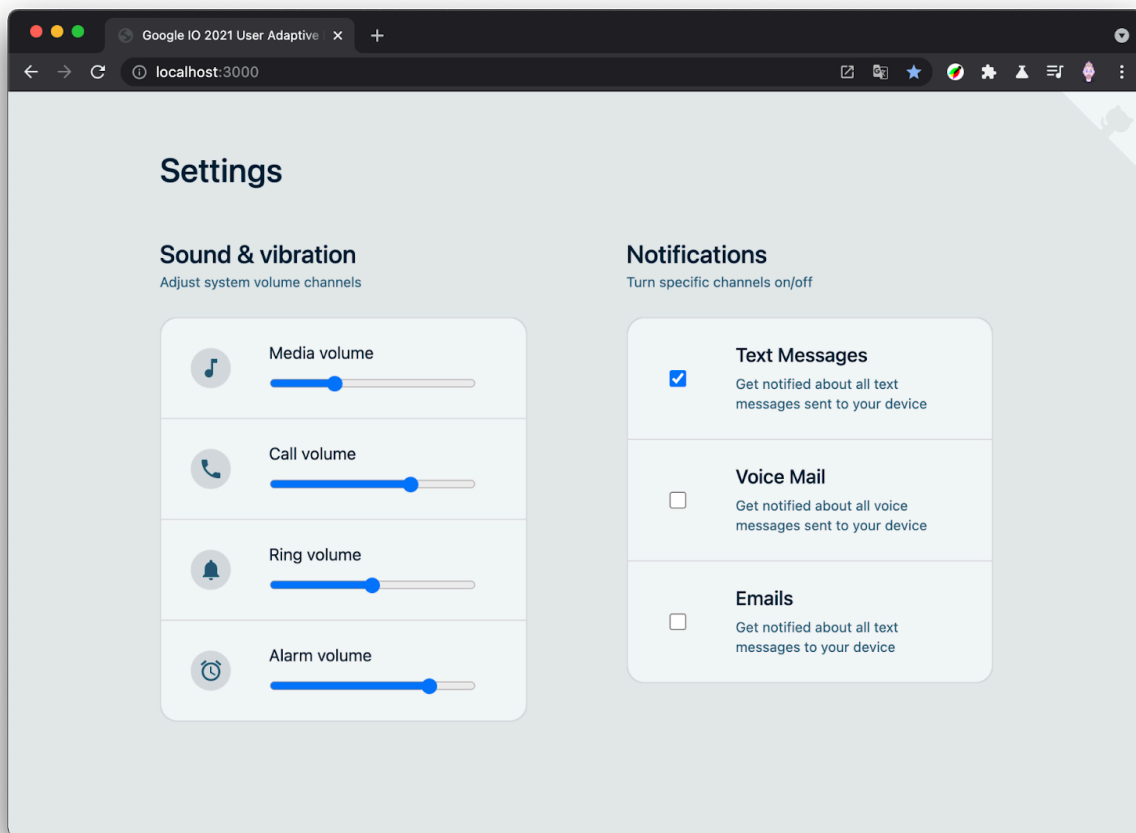
- 配合深色和淺色偏好設定
- 從品牌十六進位顏色代碼衍生色彩配置
- 使用無障礙顏色



HSL

在下一節中，您將使用 HSL 建立色彩系統，製作淺色和深色主題。這項功能是以 CSS 的核心概念 `calc()` 為基礎。

HSL 代表色調、飽和度和亮度。色調是角度，就像時鐘上的點，而飽和度和亮度則是百分比。`calc()` 可以計算百分比和角度。您可以在 CSS 中對這些百分比執行明度和飽和度計算。將色彩通道計算與自訂屬性結合，即可獲得現代化的動態色彩配置，其中變體是根據基本顏色計算而來，有助於避免在程式碼中管理大量顏色。



自訂屬性

在本節中，您將建構一組自訂屬性，供其餘樣式使用。與您先前在 `:root` 標記中設定的間距類似，您將新增顏色。

假設應用程式的品牌顏色為 `#0af`。您的第一項工作是將這個十六進位顏色值轉換為 HSL 顏色值：`hsl(200 100% 50%)`。這項轉換會以 HSL 顯示品牌色調，您可以使用 `calc()` 計算各種輔助品牌顏色。

注意：這個應用程式偏好深色色彩配置，然後會調整為淺色主題。建構第一個色彩配置時，這點非常重要，因為這會是深色主題。

本節中的每個程式碼區塊都應加入相同的 `:root` 選取器。

品牌頻道

```
:root {  
  --hue: 200;  
  --saturation: 100%;  
  --lightness: 50%;  
}
```

這三個 HSL 管道已擷取出來，並放入各自的自訂屬性中。

1. 請使用這三項屬性，並重新建立品牌顏色。

品牌

```
:root {
  --brand: hsl(
    var(--hue)
    calc(var(--saturation) / 2)
    var(--lightness)
  );
}
```

由於色彩配置預設為深色，建議您降低色彩飽和度，以便在深色表面上使用 (否則可能會造成視覺干擾或無法存取)。如要降低品牌色彩的飽和度，請照常使用色相和明度，但將飽和度除以某個數字，例如：`calc(var(--saturation) / 2)`。現在品牌顏色已正確套用至主題，但飽和度較低，可供使用。

文字

```
:root {
  --text1: hsl(var(--hue) 15% 85%);
  --text2: hsl(var(--hue) 15% 65%);
}
```

在深色主題中，閱讀文字會以品牌色調為基礎，但會從中建構出近乎白色的顏色。許多使用者會以為文字是白色，但其實是淺藍色。使用同一色調可營造和諧的設計。`--text1` 為 85% 白色，`--text2` 為 65% 白色，兩者飽和度都很低。

2. 將程式碼新增至專案後，開啟 Chrome 開發人員工具，並探索如何變更這些管道值。感受 HSL 及其通道如何相互影響。也許你喜歡更飽和或不飽和的色彩。

Surface

```
:root {
  --surface1: hsl(var(--hue) 10% 10%);
  --surface2: hsl(var(--hue) 10% 15%);
  --surface3: hsl(var(--hue) 5% 20%);
  --surface4: hsl(var(--hue) 5% 25%);
}
```

文字顏色很淺，因為深色模式下的介面會是深色。如果文字顏色使用高明度值 (85% 以上)，介面會使用較低的值 (30% 以下)。表面和文字之間的明度範圍保持適當差距，有助於確保使用者能閱讀無障礙顏色。

3. 請注意，顏色從最深的灰色開始，亮度為 10%，飽和度為 10%，然後隨著顏色變淺而降低飽和度。每個新表面都比前一個輕 5%。在較淺的表面上，飽和度也會稍微降低。請嘗試將所有表面飽和度調至 10%。你比較喜歡哪一個？

淺色主題

有了指定深色主題的健全文字和介面顏色組合後，接下來請在 `prefers-color-scheme` 媒體查詢中更新顏色自訂屬性，以配合淺色主題偏好設定。

您會使用相同的技巧，在介面和文字顏色之間保持較大的亮度值差異，確保顏色對比度良好。

品牌

```
@media (prefers-color-scheme: light) {
  :root {
    --brand: hsl(
      var(--hue)
      var(--saturation)
      var(--lightness)
    );
  }
}
```

首先是品牌顏色。因此需要恢復飽和度，才能發揮最大效用。

Text

```
@media (prefers-color-scheme: light) {
  :root {
    --text1: hsl(
      var(--hue)
      var(--saturation)
      10%
    );

    --text2: hsl(
      var(--hue)
      calc(var(--saturation) / 2)
      30%
    );
  }
}
```

與深色主題的淺藍色文字顏色類似，淺色主題的文字顏色為深藍色。如果 HSL 顏色的亮度值為 10% 和 30%，就表示這些顏色很暗。

Surface

```
@media (prefers-color-scheme: light) {
  :root {
    --surface1: hsl(var(--hue) 20% 90%);
    --surface2: hsl(var(--hue) 10% 99%);
    --surface3: hsl(var(--hue) 10% 96%);
    --surface4: hsl(var(--hue) 10% 85%);
  }
}
```

這些介面顏色是第一個打破模式的顏色。到目前為止，看似相當合理且線性的內容現在已中斷。好處是您可以在程式碼中直接使用 HSL 淺色主題色彩組合，並調整明度和飽和度，建立不會太冷或太藍的淺色色彩配置。

使用色彩系統

現在顏色已定義完成，接下來要使用這些顏色。您有醒目的品牌強調色、兩種文字顏色和四種表面顏色。

- 在下列程式碼區段中，找出相符的選取器，並將顏色 CSS 新增至現有程式碼區塊。

<body>

```
body {  
  background: var(--surface1);  
  color: var(--text1);  
}
```

網頁原色是您建立的第一個介面和文字顏色，這也會將預設對比度設為最大值。現在可以開始測試淺色和深色模式切換功能！

<fieldset>

```
fieldset {  
  border: 1px solid var(--surface4);  
  background: var(--surface4);  
}
```

這是設計中的卡片狀元素。1 像素的邊框和 1 像素的間隙顏色相同，代表每個 `.fieldset-item` 後方的表面。這樣可營造和諧的視覺效果，且易於維護。

.fieldset-item

```
.fieldset-item {  
  background: var(--surface3);  
}
```

每個表單輸入內容都位於各自的介面上。希望您已瞭解這些元素如何結合，以及明度變化如何疊加。

.fieldset-item > picture

```
.fieldset-item > picture {  
  background: var(--surface4);  
}
```

這是為了展示圖示周圍的圓形而做的樣式選擇。在下一節中新增互動後，您就會明白原因。

.fieldset-item svg

```
.fieldset-item svg {  
  fill: var(--text2);  
}
```

表單中的圖示已設為使用替代文字 `--text2`。如果實心圖示比文字稍淺，設計就不會顯得太過沉重。

.fieldset-item:focus-within svg

```
.fieldset-item:focus-within svg {  
  fill: var(--brand);  
}
```

當使用者與輸入容器元素內的其中一個輸入內容互動時，這個選取器會比對輸入容器元素，並以品牌強調色醒目顯示 SVG。這樣一來，使用者與輸入內容互動時，相關圖示就會醒目顯示，提供良好的表單 UX 回饋。

<small>

```
small {  
  color: var(--text2);  
}
```

這是小字。與標題和段落 (主要內容) 相比，應略為柔和。

深色表單控制項

```
:root {  
  color-scheme: dark light;  
}
```

這個最後的潤飾可告知瀏覽器，這個頁面同時支援深色和淺色主題。瀏覽器會提供深色表單控制項做為回報。

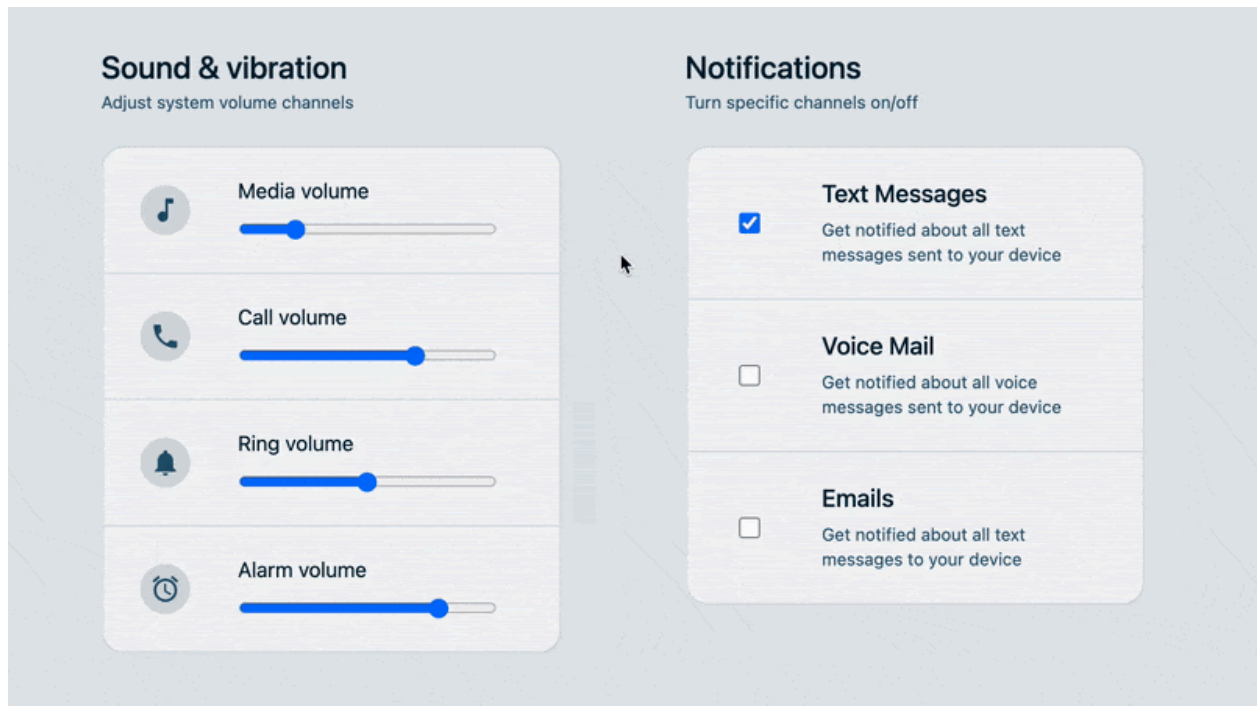
步驟 6 · 約 7 分鐘

6. 自動調整動畫

Git 分支版本：`animations`

完成本節後，「設定」頁面將：

- 配合使用者的動態偏好設定
- 回應使用者互動



減少動態效果與無動態效果的比較

作業系統中找到的動作使用者偏好設定不提供無動畫的值。這個選項可減少動作。對於偏好減少動作的使用者來說，淡入淡出動畫、色彩轉換等效果仍有其必要性。

在這個設定頁面中，螢幕上的動作不多，這項動作更像是縮放效果，彷彿元素正朝向使用者移動。調整 CSS 程式碼以配合減少動作非常簡單，因此您可以減少縮放比例轉換。

互動風格

<fieldset>

```
fieldset {  
  transition: box-shadow .3s ease;  
}  
  
fieldset:focus-within {  
  box-shadow: 0 5px 20px -10px hsl(0 0% 0% / 50%);  
}
```

當使用者與其中一個 `<fieldset>` 類卡片元素的輸入內容互動時，系統會新增升起效果。介面會將元素往前推，協助使用者專注於情境表單群組。

.fieldset-item

```
.fieldset-item {  
  transition: background .2s ease;  
}  
  
.fieldset-item:focus-within {  
  background: var(--surface2);  
}
```

使用者與輸入內容互動時，特定項目層的背景會變更為醒目顯示的介面顏色，藉此吸引使用者注意，並表示系統正在接收輸入內容。在大多數情況下，不需要減少色彩轉換。

.fieldset-item > picture

```
@media (prefers-reduced-motion: no-preference) {  
  .fieldset-item > picture {  
    clip-path: circle(40%);  
    transition: clip-path .3s ease;  
  }  
  
  .fieldset-item:focus-within picture {  
    clip-path: circle(50%);  
  }  
}
```

如果使用者沒有偏好設定，請使用以下 `clip-path` 動畫。第一個選取器和樣式會將圓形剪輯路徑縮小 10%，並設定一些轉換參數。第二個選取器和樣式會等待使用者與輸入內容互動，然後放大圖示的圓圈。如果一切正常，就會出現細微但整齊的效果。

步驟 7 · 約 0 分鐘

7. 恭喜

Git 分支版本：`complete`

恭喜！您已成功建構可適應使用者的介面！

您現在已瞭解重要步驟，可建立適用於各種使用者情境和設定的介面。
